SECTION 1 — DSA (3 PROBLEMS)

Problem 1: Message Delivery System

Problem Description

You are given a vector of strings **messages**, representing a sequence of incoming messages, and an integer **k**, representing a time window in seconds. Determine whether each message should be delivered.

A message must **not** be delivered if the same message has already appeared within the previous **k** seconds. Otherwise, it can be delivered.

Return a vector of strings representing the delivery status of each incoming message:

• **"true"** — the message is delivered.
• **"false"** — the message is not delivered.

Function Signature

```
vector<string> messageDelivery(vector<string>& messages, int k);
```

Example

```
messages = {"hello", "hi", "hello", "welcome", "hi", "hello", "hello"};
k = 3;
```

Output

```
{"true", "true", "false", "true", "false", "true", "false"}
```

Explanation

1. The first *hello* is new, so it is delivered.
2. The first *hi* is new, so it is delivered.
3. *hello* appeared within the previous 3 seconds, so it is blocked.
4. *welcome* is new, so it is delivered.
5. *hi* appeared within the restricted window, so it is blocked.
6. The previous *hello* occurrence is now outside the restricted time window, so it is delivered.
7. *hello* was delivered recently, so it is blocked.

Constraints

• $1 \le$ messages.length $\le 10^5$
• $1 \le k \le 10^9$
• Each message is a non-empty string.

Expected Output

Return a vector of strings containing `"true"` or `"false"` for every incoming message.

Suggested topics: Hash map, last-seen index / timestamp, sliding window. The example assumes each vector position corresponds to one second.

Problem 2: Beautiful Graph — Codeforces 1093D

Problem Description

You are given an undirected, unweighted graph with **n** vertices and **m** edges. Assign exactly one number from {1, 2, 3} to every vertex. The graph is beautiful if, for every edge, the sum of the numbers assigned to its endpoints is odd.

Calculate the number of valid assignments, modulo **998244353**.

Input Format

The first line contains **t**, the number of test cases. For each test case, read **n** and **m**, followed by **m** undirected edges `(u, v)`.

Output Format

For each test case, print the number of valid assignments modulo 998244353.

Constraints

• $1 \le t \le 3 \times 10^5$
• $1 \le n \le 3 \times 10^5$
• $0 \le m \le 3 \times 10^5$
• The sums of n and m over all test cases are each at most 3×10^5.

Sample Input

```
2
2 1
1 2
4 6
1 2
1 3
1 4
2 3
2 4
3 4
```

Sample Output

```
4
0
```

Problem link: https://codeforces.com/contest/1093/problem/D

Suggested topics: BFS/DFS, bipartite graph checking, connected components, modular arithmetic.

Problem 3: Minimum Execution Time — Process Scheduling

The following screenshots are attached from the sample test. They contain the original statement, scheduling rules, example, function description, and sample test details.

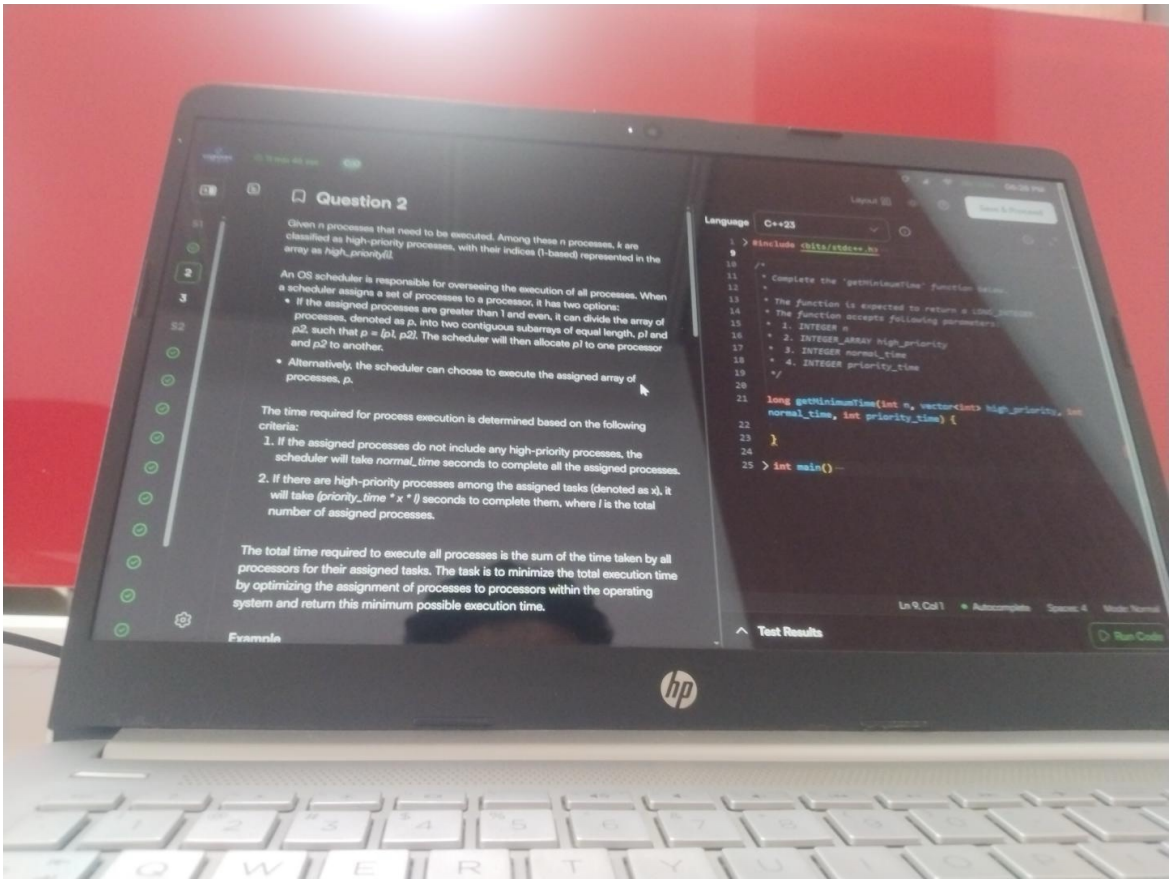
Key details visible in the statement:

• There are **n** processes; **k** of them are high-priority, identified by their indices in `high_priority`.
• A scheduler can assign a contiguous segment of the process array to a processor, or execute the assigned array without splitting it.
• If a segment has no high-priority process, its execution time is `normal_time`.
• If a segment contains high-priority processes, its execution time is `priority_time × (x + 1)`, where `x` is the number of high-priority processes in that segment.
• Minimize the sum of execution times over the chosen assignments.

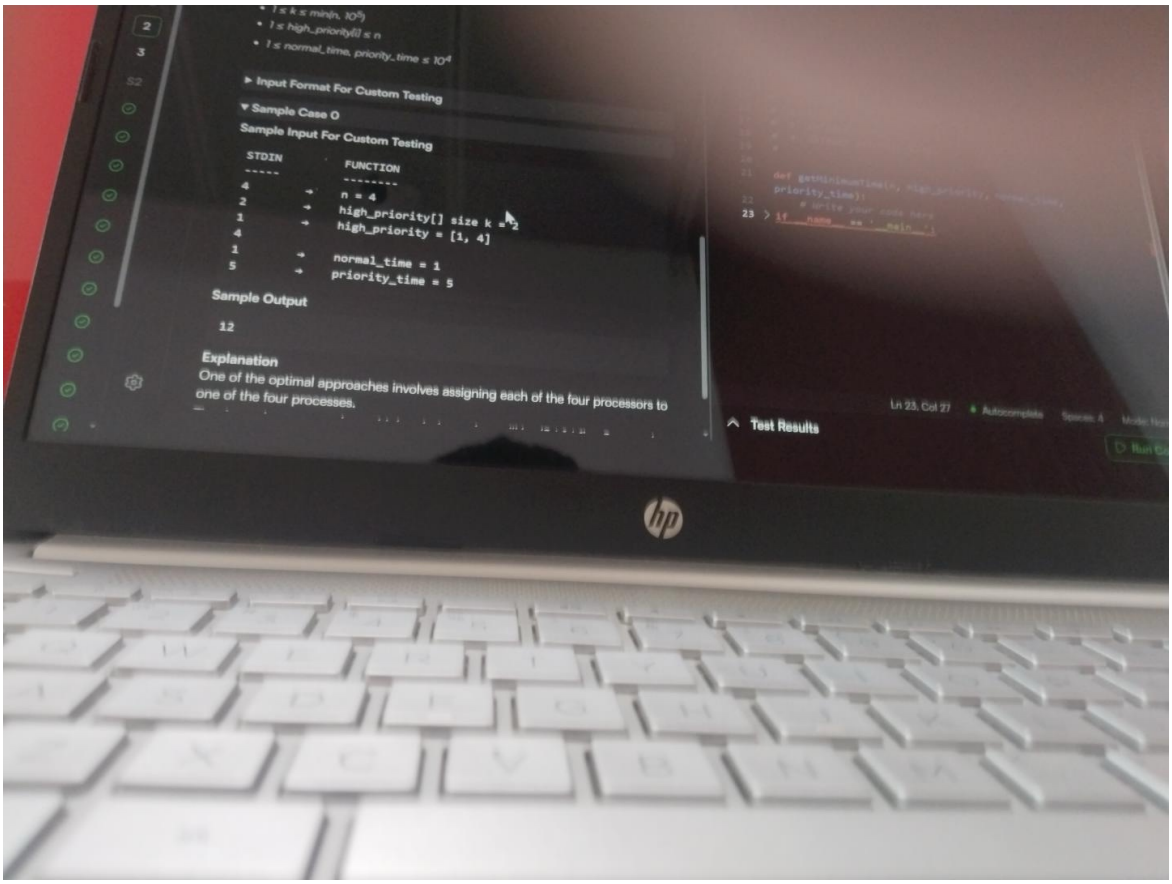
Function signature: `getMinimumTime(n, high_priority, normal_time, priority_time)`

Original screenshots

Screenshot 1

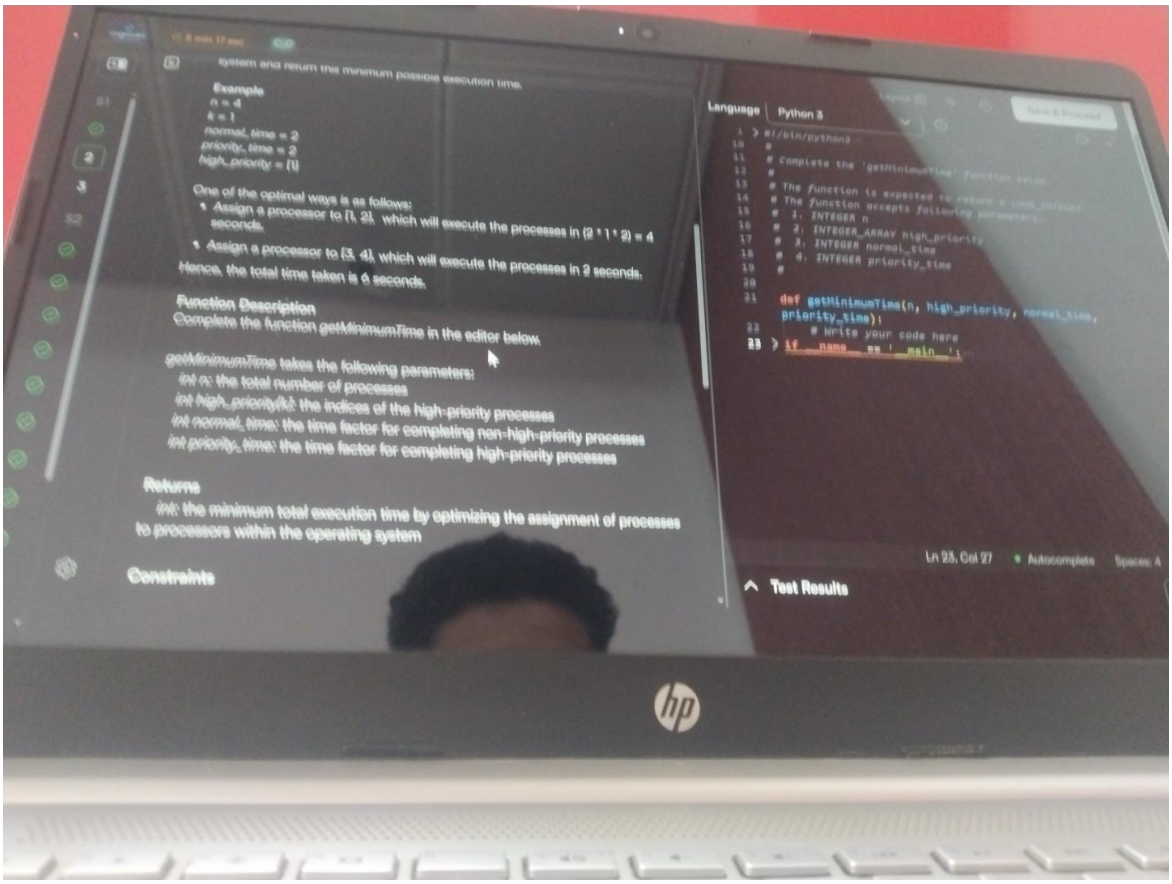


Screenshot 2


Input Format For Custom Testing
Sample Case 0
Sample Input For Custom Testing
STDIN
FUNCTION
4
2
1
4
1
5
n = 4
high_priority[] size k = 2
high_priority = [1, 4]
normal_time = 1
priority_time = 5
Sample Output
12
Explanation
One of the optimal approaches involves assigning each of the four processors to one of the four processes.
Test Results

Problem 3 — Original screenshots (continued)

Screenshot 3



SECTION 2 — MCQs

18 MCQs based on Generative AI, Large Language Models (LLMs), and Retrieval-Augmented Generation (RAG).